



Profa. Anelise Munaretto

Redes sem Fio — Ciência da Computação

2026

Ao final desta aula, você será capaz de:

- **descrever** o ciclo agente–ambiente do RL e **diferenciar** aprendizado centralizado de *federated learning*;
- **distinguir** aprendizagem supervisionada, não-supervisionada, por reforço e profunda;
- **aplicar** a equação de Bellman e a atualização Q-learning em alocação de recursos;
- **explicar** o uso de LSTM na predição de mobilidade e *handover*;
- **avaliar** desafios de dados, privacidade, confiabilidade e padronização de ML em redes (3GPP RM-NET, O-RAN).

Pré-requisitos

Aulas 7 (5G e redes móveis) e 10 (drones, satélites e NTN).

Crise de complexidade

Redes 5G/6G têm centenas de parâmetros ajustáveis; otimização manual é inviável. IA/ML oferece soluções para:

- predição de tráfego e mobilidade
- alocação dinâmica de recursos (espectro, potência, beam)
- detecção de anomalias e falhas
- gerenciamento autônomo (*zero-touch*)
- otimização cross-layer

Aprendizagem Supervisionada

- classificação de interferência
- predição de SNR/CQI
- detecção de tipo de tráfego

Aprendizagem Não-Supervisionada

- agrupamento de padrões de mobilidade
- compressão de CSI

Aprendizagem por Reforço (RL)

- *resource block allocation*
- escalonamento de pacotes
- controle de admissão

Aprendizagem Profunda (DL)

- *channel estimation* com autoencoders
- beamforming com CNN
- predição de séries temporais (LSTM)



Figura 1: ciclo agente–ambiente do aprendizado por reforço aplicado à alocação de recursos.

- Exemplo: alocação de RBs em 5G usando Deep Q-Learning (DQN)
- Estado: filas CQI, buffer; Ação: RBs por usuário; Recompensa: throughput $\times$ justiça (Jain)

- **Problema:** antecipar handover para evitar perda de pacotes
- **Método:** LSTM treinado com sequências de RSSI/RSRP
- **Entrada:** série temporal de 10 medições de potência (5 vizinhos)
- **Saída:** célula-alvo e tempo estimado de handover
- **Resultado:** redução de 40% em falhas de handover (trabalhos reais)

Implementação

Python + TensorFlow/Keras, treino offline com dados de drive-test, inferência no MEC ou no próprio UE.

Dados

- coleta de dados rotulados (CSI, tráfego)
- privacidade (GDPR, nuvem vs. borda)
- distribuição não-estacionária (mobilidade)

Complexidade

- inferência em tempo real (latência)
- tamanho do modelo (memória do dispositivo)
- *federated learning* para escalar

Confiabilidade

- modelos precisam generalizar
- interpretabilidade (explainable AI)
- robustez a ataques adversários

Padronização

- 3GPP Rel. 18: RM-NET (ML para RAN)
- O-RAN Alliance: rApp e xApp com ML
- ITU-T ML5G

Problema: um agente RL decide a alocação de um RB. A ação atual não gera recompensa imediata ($r = 0$), mas leva a um estado em que a melhor ação futura renderá $r = 10$ (estado terminal). Considere $\gamma = 0,9$ e partida com $Q(s, a) = 0$.

❶ **Valor-alvo (Bellman):**

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a') = 0 + 0,9 \times 10 = 9.$$

❷ **Atualização Q-learning** com $\alpha = 0,1$:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] = 0 + 0,1 (9 - 0) = 0,9.$$

❸ **Convergência:** repetindo a mesma experiência, $Q(s, a)$ se aproxima assintoticamente de 9.

Interpretação

O fator $\gamma = 0,9$ desconta o valor futuro: uma recompensa de 10 a um passo de distância vale $0,9 \times 10 = 9$ hoje.

- **ML em redes:** predição de tráfego/mobilidade, alocação dinâmica de espectro/potência/beam, detecção de anomalias, *zero-touch*.
- **Paradigmas:** supervisionado (classificação), não-supervisionado (agrupamento), por reforço (alocação) e profundo (LSTM, CNN, autoencoders).
- **RL/DQN:** agente aloca RBs com base em estado (CQI, buffer) e recompensa (throughput $\times$ justiça de Jain); DQN usa redes neurais como aproximador de Q .
- **LSTM:** predição de *handover* a partir de séries de RSSI/RSRP (redução de $\approx 40\%$ de falhas).
- **Desafios:** dados rotulados, privacidade (GDPR), não-estacionariedade, latência de inferência e robustez.
- **Padronização:** 3GPP Rel. 18 RM-NET, O-RAN (rApp/xApp) e ITU-T ML5G; *federated learning* para escalar com privacidade.

1 No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

① No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

Gabarito: (c)

② Uma LSTM é mais adequada para:

- (a) predição de séries temporais, como RSSI/RSRP
- (b) ordenação de pacotes em filas
- (c) compressão de imagens de beamforming

❶ No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

Gabarito: (c)

❷ Uma LSTM é mais adequada para:

- (a) predição de séries temporais, como RSSI/RSRP
- (b) ordenação de pacotes em filas
- (c) compressão de imagens de beamforming

Gabarito: (a)

❸ A principal diferença do *federated learning* para o treino centralizado é:

- (a) usar apenas redes neurais profundas
- (b) treinar com dados locais e agregar apenas os modelos/gradientes
- (c) dispensar a camada de rede física

❶ No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

Gabarito: (c)

❷ Uma LSTM é mais adequada para:

- (a) predição de séries temporais, como RSSI/RSRP
- (b) ordenação de pacotes em filas
- (c) compressão de imagens de beamforming

Gabarito: (a)

❸ A principal diferença do *federated learning* para o treino centralizado é:

- (a) usar apenas redes neurais profundas
- (b) treinar com dados locais e agregar apenas os modelos/gradientes
- (c) dispensar a camada de rede física

Gabarito: (b)

❹ Com $\gamma = 0,9$, $r = 0$ e $\max Q(s', a') = 10$, a equação de Bellman resulta em $Q(s, a) =$

- (a) 9

(1) 10

❶ No aprendizado por reforço, o agente obtém do ambiente:

- (a) apenas o rótulo correto da ação
- (b) apenas a ação executada
- (c) o novo estado e a recompensa

Gabarito: (c)

❷ Uma LSTM é mais adequada para:

- (a) predição de séries temporais, como RSSI/RSRP
- (b) ordenação de pacotes em filas
- (c) compressão de imagens de beamforming

Gabarito: (a)

❸ A principal diferença do *federated learning* para o treino centralizado é:

- (a) usar apenas redes neurais profundas
- (b) treinar com dados locais e agregar apenas os modelos/gradientes
- (c) dispensar a camada de rede física

Gabarito: (b)

❹ Com $\gamma = 0,9$, $r = 0$ e $\max Q(s', a') = 10$, a equação de Bellman resulta em $Q(s, a) =$

- (a) 9

(b) 10

Q-learning em Python/gymnasium

- Implementar Q-learning para alocação de canal/RB em um ambiente simplificado (ex.: FrozenLake ou um env customizado).
- Variar α e γ e observar a curva de recompensa acumulada e a convergência da tabela Q .
- Extensão: substituir a tabela por uma rede neural (DQN) usando PyTorch/TensorFlow.

Demonstração de *federated learning*

- Usar tensorflow-federated (TFF) para treinar um classificador com clientes simulados.
- Comparar precisão e custo de comunicação entre treino centralizado e federado.

- C. Zhang et al., “Deep learning in mobile and wireless networking: A survey”, *IEEE Commun. Surv. Tut.*, 2019
- N. C. Luong et al., “Applications of deep reinforcement learning in communications and networking”, *IEEE Commun. Mag.*, 2019
- 3GPP TR 38.817, *Study on RAN-centric data collection and utilization (RAN-CU)*
- O-RAN Alliance, *O-RAN Working Group 2: AI/ML Workflow Description*
- ITU-T Y.3172, *Architectural framework for ML in future networks*

- R. Sutton & A. Barto, *Reinforcement Learning: An Introduction*, MIT Press — referência clássica para RL, Bellman e Q-learning.
- C. Zhang et al., “Deep learning in mobile and wireless networking: A survey”, *IEEE Commun. Surv. Tut.*, 2019 — panorama geral de DL aplicado a redes.
- N. C. Luong et al., “Applications of deep reinforcement learning in communications and networking”, *IEEE Commun. Mag.*, 2019 — DQN aplicado a alocação de recursos.
- O-RAN Alliance, *WG2: AI/ML Workflow Description* — como o ML entra na RAN (rApp/xApp).
- 3GPP TR 38.817 (RAN-CU) — coleta e uso de dados centrados na RAN para ML.

Dica de estudo

Reimplemente o exercício de Bellman em Python e depois generalize para DQN; isso consolida o ciclo agente–ambiente.